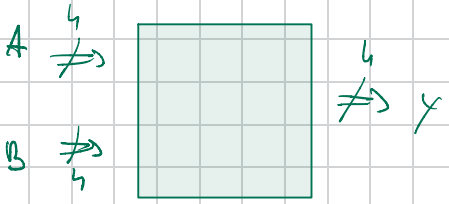
fps : 60 → 16 ms.

⇒ Conception : ASIC ou fpga

⇒ modélisation : langage

vérification :

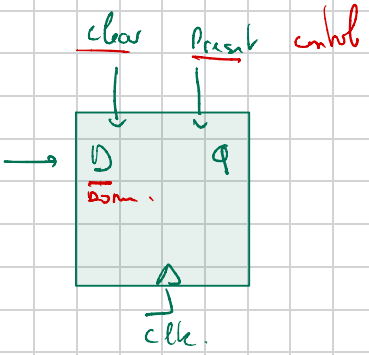
additionneur :




```
Entity add_4bit is
    Port ( A, B : In  std_logic_vector (3 downto 0);
           Y    : out std_logic_vector (3 downto 0));
END add_4bit;
```

32 / 31 / 31 / 32

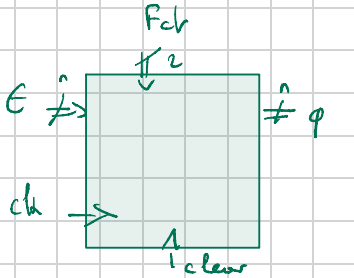
Registre




```
Entity Basc_D is
    Port ( D    : In  std_logic_vector.
           clear, Preset : In std_logic_vector
           ck   : In  std_logic_vector
           Q    : out std_logic_vector );
END Basc_D ;
```

Registre / Registre

Registre à décalage

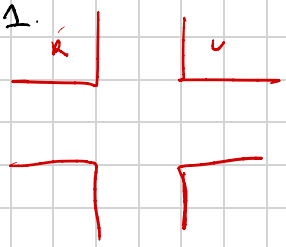


Fct		Q	
0	0	Mem	
0	1	E	
X2	1	0	Shf left	décalage
X3	1	1	Shf right	

```
Entity Shift_Reg_32b is
    Port ( E    : In  std_logic_vector (31 downto 0);
           clk  : In  std_logic;
           clear : In std_logic;
           Fct  : In  std_logic_vector (1 downto 0);
           Q    : out std_logic_vector (31 downto 0));
END Shift_Reg_32b ;
```

Buffer → contrôle.

Exo 1.



type Fonction_etat is.

($F_1 R F_2 U$, $F_1 R F_2 0$, $F_1 R F_2 R$, $F_1 V F_2 R$, $F_1 0 F_2 R$, $F_1 R F_2 R$)

→ Subtype index Entier range 1 to 15.

S ⇐ A + Cst

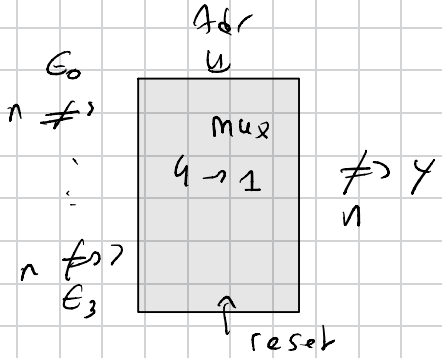
type RAM 16kx32 IS ARRAY.
0 to 16 000 of std_logic_vector (31 downto 0)

type bloc is ARRAY
(0 to 15, 0 to 15) of std_logic_vector
(23 down to 0);

type matbloc IS ARRAY.
(0 to 3, of bloc,

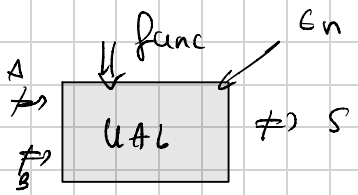
Cas MUX :

Y <= (other => 0') when Reset = '1' else
E_0 when (Adr = "00") else
E_1 01
E_2 10
E_3 11 else
other => 'U'



with adr SELECT Y <= E_0 when "00";
E_1 "01"
E_2 "10"
E_3 "11"
(other => 'U') when others;

Pour le reset possible

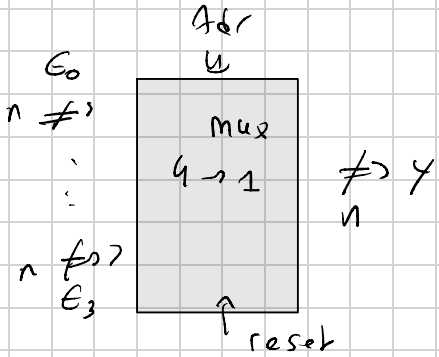


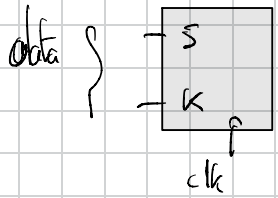
func	S
0 0	And
0 1	or
1 0	+
1 1	−

S ⇐ S when En = '0' else
A and B when (func = "00") ...
or
+
−
others ⇒ 'U'

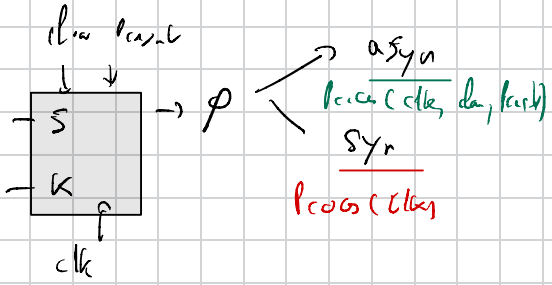
Process (ads, Reset, E_0, E_1, E_2, E_3)

Process (clk)





$Q^+ \Leftarrow S\bar{Q} + \bar{R}Q$

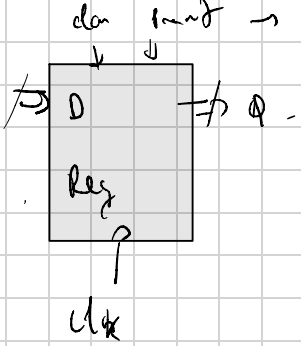


→ Système Combinatoire :
Process (toutes les entrées)

Système synchrone (avec une horloge)

asynchrone (indépendant de l'horloge)

Process (l'horloge, entrées asynchrones).

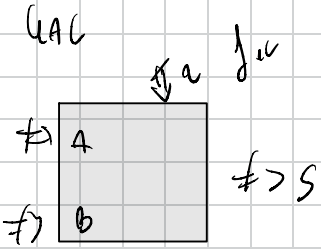


asynchron.
Synchron

```
if ( clear = '1' ) then  Q <= (others => '0');
elsif ( preset = '1' ) then Q <= (others => '1');
elsif ( CK'event and clk = '1' ) then Q <= D;
end if;
```

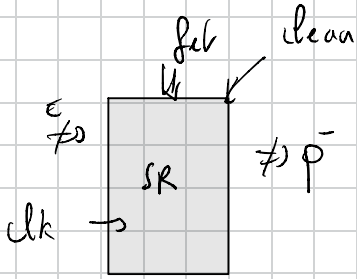
```
if (clk'event and clk = '1') then
elsif ( clear = '1' ) then Q <= (others => '0');
elsif ( Preset = '1' ) then Q <= (others => '1');
else Q <= 1)
end if;
end if;
```

instruction en cours
execution
→ plutôt choisir
Buffer : sort donnée




```
Process (A, B, sel)
Begin
CASE sel IS
    when "00" => S <= A and B;
                 "01"   A or B
                 "10"   A + B
                 "11"   A - B
    when others => NULL
                   S <= (others => '0');
end case;
end process;
```

Shift Reg



fct	S
00	$\varphi \in E$
01	Shift L
10	R.
11	effacer mémoire

Process (clk, clean)

BEGIN

"asynchrone clean" ou 1 condition dans "if"

if (clean = '1') then $\varphi \leftarrow$ (others => '0')

elsif (clk'event and clk = '1')

then case fct :

when "00" => $\varphi \leftarrow E$

— "01" => $\varphi \in \varphi$;

— "10" => $\varphi \in \varphi$ SR '1' ;

when others => NULL;

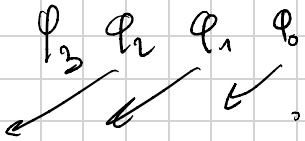
end case ;

end if ;

end Process.

$\varphi(i) \leftarrow \varphi(i-1)$:

donc décalage à gauche.



C / Python

for i [0, 3].

S(i) <= A(i) + B(i)

TAG :

A(0), B(0)	S(0)
A(1), B(1)	S(1)

VHDL

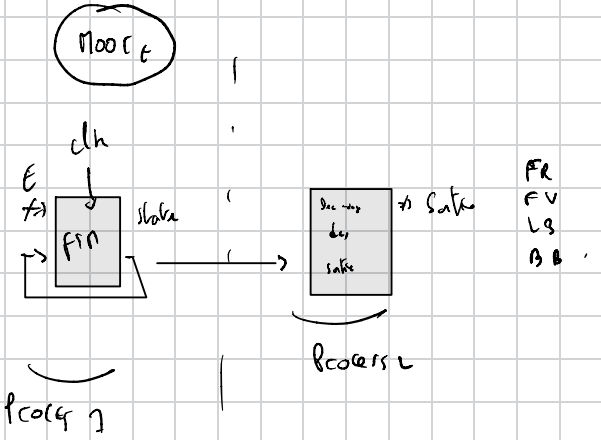
S(0) <= A(0) + B(0)

S(1) <= A(1) + B(1).

:

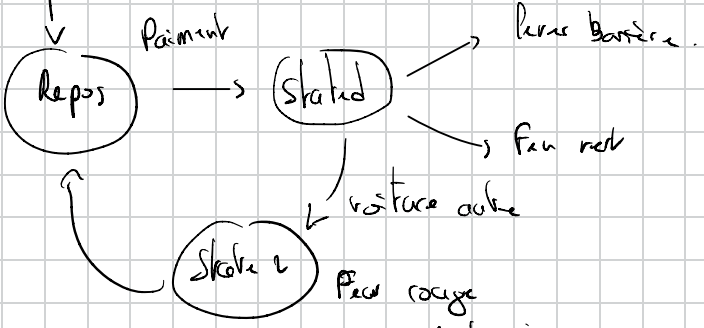
S <= A + B.

Combien Process



Contrôl :

Feu rouge.



P1 : Process (clk ...)

BEGIN

if (clk 'EVENT and clk = '1').

then case etat is

when Repos => if (PV = '1' & PA = '1') then etat <= start 1;

end if;

when start 1 => if (FV = '1') then etat <= start 2 ;

end if;

when start 2 => etat <= Repos

when others => NULL.

end case ;

end if ; end process

Process (Etat ...).

Begin :

Case Etat is

when Repos => FR <= '1' ; FV <= '0', BL <= '0' ; BB <= '0'

State 1 => FR <= '0' ; FV = '1', BL <= '1', BB <= '0'

State 2 => FR <= '1' ; FV <= '0', BL = '0', BB <= '1'.

end Case

end process.

Valeur Sortie.

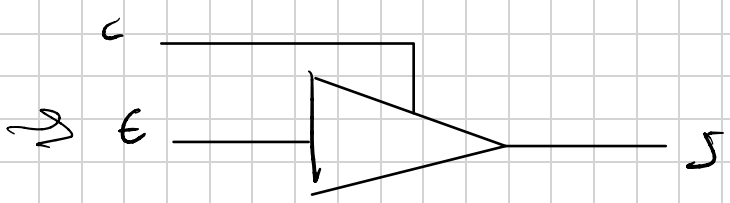
Intersection : Rouleau

avec : V = 1 allumé.

0 => 10s.

R =>

Feu		$Etat_0$	$Etat_1$	$Etat_2$
	R	1	0	0
	V	!	1	0
	O	0	0	1



C	S
0	High (Z)
1	E

8 bits :

64 32 16 8 4 2 1

Process (C, E).

si C = 1 → S = E

si C = 0 → S = Z

C => in std_logic.

S => in std_logic_vector (7 .. 0).

8 bits.

tp 1 VHDL:

8 → 2

1000

0111

00000

0101

1111

